

Troisième année de Licence MIASHS

TP R – Compléments de mathématiques 1¹

Julien GREPAT²

Contents

I	TP1 – Analyse de Fourier	3
1	Les complexes	3
2	Séries de Fourier	3
2.1	Intégrer une fonction	3
2.2	La fonction	4
2.3	Coefficients de la série de Fourier	5
2.4	Convergence de la série de Fourier	5
3	Travail à rendre	6
II	TP2 – Transformée de Fourier et traitement du signal	7
4	Transformée de Fourier	7
4.1	Le problème	7
4.2	Corrigé	7
5	Traitement du signal	7
5.1	Échantillonnage	8
5.2	Classification discret–continu	8

¹Reproduction et diffusion interdite sans l'accord de l'auteur

²Contact : julien.grepat@univ-grenoble-alpes

5.3	Transformée de Fourier Discrète – Fast Fourier Transform (FFT)	10
5.4	Le signal	11
5.5	Fast Fourier Transform	11
5.6	Filtrage du signal	12
III	TP3 – Équations différentielles	13
6	Librairie R	13
7	Prise en main	13
7.1	L'équation	13
7.2	Mise en mémoire de l'équation	13
7.3	Champ de directions	13
7.4	Solution de $\mathcal{C}$	14
7.5	Variations de y_0	14
7.6	Solutions issues de $y_0 = 1$ en fonction des valeurs de a	15
8	Les modèles de croissance de population	15
8.1	Modèle de Malthus	16
8.2	Le modèle logistique de Verhulst	17
9	Travail à rendre	19
9.1	Pêche sans quota	19
9.2	Le modèle de pêche par quota	20

Part I

TP1 – Analyse de Fourier

1 Les complexes

Prenons deux complexes

$$\begin{array}{l} z_1 = 1 + 3i \\ z_2 = 3i + 4 \end{array}$$

Miraculeusement, R comprend tout de suite que nous lui parlons de complexes. Attention, la magie de R a ses limites, il identifiera le seul `i` comme la variable nommée `i`³. Pour obtenir l'unité complexe i , on tapera `1i`.

R se débrouille très bien des opérations algébriques classiques

$$z_1 + z_2 \quad z_1 + 3 * z_2 \quad z_1 * z_2 \quad z_1^2 \quad z_1 / z_2$$

Nous pourrions observer les principales fonctions nécessaires aux complexes (faire afficher les résultats):

$$a = \text{Re}(z_1) \quad b = \text{Im}(z_1) \quad \theta = \text{Arg}(z_1) \quad r = \text{Mod}(z_1)$$

Il est possible de saisir un complexe directement avec sa partie réelle et sa partie imaginaire :

$$\text{complex}(\text{real} = a, \text{imaginary} = b)$$

On peut vérifier alors la cohérence de la forme trigonométrique

$$r * (\cos(\theta) + \sin(\theta) * 1i)$$

(On fera attention à bien représenter i par `1i`), et de la forme exponentielle

$$r * \exp(\theta * 1i)$$

2 Séries de Fourier

2.1 Intégrer une fonction

On se propose de calculer l'intégrale

$$\int_0^1 \arctan(x) dx.$$

³On pourra mettre en mémoire `i=1i`

- (i) On peut calculer cette intégrale *à la main*. Par intégration par partie, trouver que la primitive de $\arctan(x)$ est

$$x\arctan(x) - \frac{1}{2}\ln(x^2 + 1).$$

Cette question est facultative, loin de l'objectif du TP et vous occupera une fois celui-ci terminé... Bon courage !

- (ii) Le logiciel R sait calculer des approximations des intégrales :

```
f<-function(x){ atan(x) }
integrate(f,lower=0,upper=1) .
```

Notons :

- `integrate` renvoie la valeur de l'intégrale et une erreur d'approximation. Si nous ne souhaitons que la valeur numérique de l'intégrale :

```
integrate(f,lower=0,upper=1)$ value
```

- `integrate` ne calcule pas les intégrales s'il y a des complexes dans la fonction. Nous pourrions utiliser l'astuce

```
ref=function(x){Re(f(x))}
imf=function(x){Im(f(x))}
integrate(ref,0,1)$ value+1i*integrate(imf,0,1)$ value
```

dès lors que l'intégrale est linéaire.

- `integrate` ne supporte pas d'opération de fonction comme argument.

L'enjeu du paragraphe est d'observer la convergence de la série de Fourier d'une fonction en ses points de discontinuité.

2.2 La fonction

On souhaite étudier la fonction 2-périodique définie sur l'intervalle de référence $[-1, 1]$ par

$$f(x) = \begin{cases} -1 & \text{si } x \leq 0 \\ 1 & \text{si } x > 0 \end{cases}.$$

On se restreindra à l'intervalle principal $[-1, 1]$ pour éviter toute technicité. On peut mettre `f` en mémoire

```
f=function(x){2*as.numeric(x>0)-1}
```

Traçons `f` dans une fenêtre un peu plus large

```
plot(f,xlim=c(-1,1),ylim=c(-2,2))
```

Naturellement, $f(0)=-1$, il ne devrait pas y avoir de barre verticale au milieu du tracé.

2.3 Coefficients de la série de Fourier

Donnons une fonction `coeff` qui pour une fonction `f` et un entier relatif `n` calcule le coefficient

$$c_n = \frac{1}{T} \int_{-1}^1 f(x) e^{-\frac{2\pi}{T}inx} dx,$$

autrement dit

$$c_n = \frac{1}{2} \int_{-1}^1 f(x) e^{-\pi inx} dx.$$

Lisez, implémentez, commentez le code suivant

```
coeff=function(f,n){  
  e=function(x,n){exp(pi*1i*n*x)}  
  g=function(x){f(x)*Conj(e(x,n))}  
  reg=function(x){Re(g(x))}  
  img=function(x){Im(g(x))}  
  0.5*integrate(reg,-1,1)$ value+0.5i*integrate(img,-1,1)$ value  
}
```

On aura reconnu le n ème coefficient de la série Fourier de f .

On peut tester nos coefficients pour quelques valeurs de n positives et négatives. On observe que

- puisque f est réelle, les coefficients c_n et c_{-n} sont conjugués l'un, l'autre.
- puisque la fonction f est impaire, sa décomposition se fait sur des fonctions trigonométriques impaires (des sinus) et donc les coefficients sont imaginaires purs.

On vous laisse y réfléchir...

2.4 Convergence de la série de Fourier

La série de Fourier

$$S_N = \sum_{k=-N}^N c_k e^{\frac{2\pi}{T}ikx}.$$

converge en tant que fonction vers f aux points de dérivabilité et vers

$$\frac{f(x_0^-) + f(x_0^+)}{2}$$

aux points de discontinuité, ici en 0,1 et -1. Observons ce phénomène. Pour commencer, créons les fonctions $S_N(x)$.

```
SN=function(f,N,x){  
  S=coeff(f,0)  
  for(n in 1:N){  
    S=S+coeff(f,n)*exp(pi*1i*n*x)+coeff(f,-n)*exp(-pi*1i*n*x)  
  }  
  S  
}
```

On prendra une discrétisation

```
discr=seq(-1,1,0.01)
```

et on tracera la série de Fourier pour les premières valeurs de N :

```
lines(discr,SN(f,1,discr))  
lines(discr,SN(f,5,discr))  
lines(discr,SN(f,9,discr),col="blue")  
lines(discr,SN(f,20,discr),col="red")  
etc...
```

3 Travail à rendre

On considère la fonction 2π –périodique définie sur $[-\pi, \pi]$ par

$$f(x) = \alpha x + 1,$$

où α est un réel strictement compris entre 1 et 2 que vous choisirez arbitrairement à 3 chiffres. On vous demande d’adapter les fonctions précédentes pour obtenir le graphe de la dernière partie sur $[-\pi; \pi]$. Vous l’enverrez alors par mail cosigné par votre binôme (un mail par binôme) à l’adresse julien.grepat@univ-grenoble-alpes.fr dans un fichier .pdf. Le graphe portera la valeur de α . Vous pourrez utiliser la commande

```
title("alpha=1,...")
```

Part II

TP2 – Transformée de Fourier et traitement du signal

4 Transformée de Fourier

4.1 Le problème

On considère la fonction

$$f(x) = \mathbb{I}_{]-1;1[}(x).$$

On souhaite obtenir et tracer la transformée de Fourier $\hat{f}$ définie par

$$\hat{f}(y) = \frac{1}{2\pi} \int f(x) e^{-ixy} dx.$$

On vous laisse toute latitude pour ce travail. Un corrigé suit.

4.2 Corrigé

```
f=function(x){as.numeric(abs(x)<1)}  
e=function(x,y){exp(1i*x*y)}  
tf=function(y){  
  g=function(x){f(x)*e(x,y)}  
  reg=function(x){Re(g(x))}  
  img=function(x){Im(g(x))}  
  1/(2*pi)*(integrate(reg,-100,100)$value+1i*integrate(img,-100,100)$value)  
}
```

Il reste à tracer notre transformée de Fourier. Malheureusement, elle calcule les valeurs de $\hat{f}$ terme à terme, et non dans une liste. Il faut prendre quelques précautions pour tracer.

```
discr=seq(-20,20,0.1)  
ltf=discr  
for(i in 1:length(discr)){ltf[i]=tf(discr[i])}  
plot(discr,ltf,type="l")
```

5 Traitement du signal

Les outils tels que la transformée de Fourier, ou les séries du même nom, ne sont adaptées qu'au signaux formels définis par une fonction. La réalité est autre !

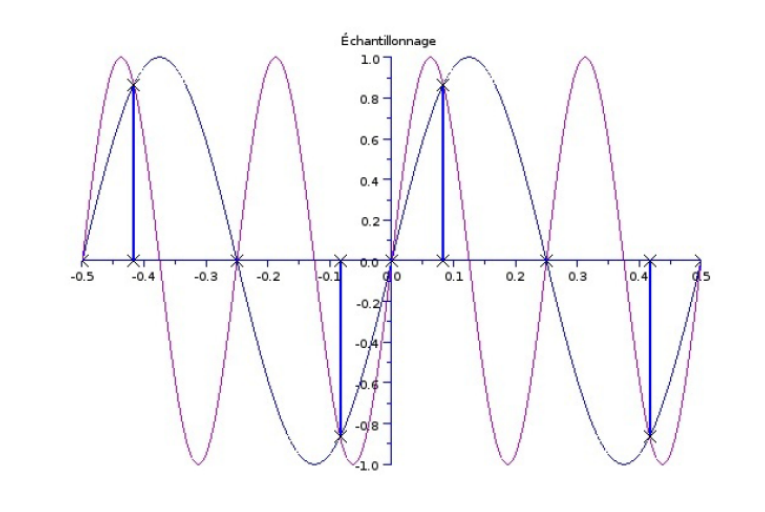
Nous vous proposons d'investiguer un algorithme usité dans les cas concrets de signaux échantillonnés.

5.1 Échantillonnage

Le traitement numérique des données (téléphonie, CD audio etc ...) ne peut se faire que sur des données finies.

Il faut donc représenter ces données par une suite de valeurs numériques ponctuelles prélevées régulièrement ou irrégulièrement. Un tel prélèvement est appelé échantillonnage.

Il est clair que l'échantillonnage visible sur la figure suivante ne permet pas de distinguer les deux signaux.



Par exemple, si $f(t) = \sin(2\pi x)$ et

$$g(t) = \sin\left(2\pi\left(\frac{1}{T} + 1\right)x\right),$$

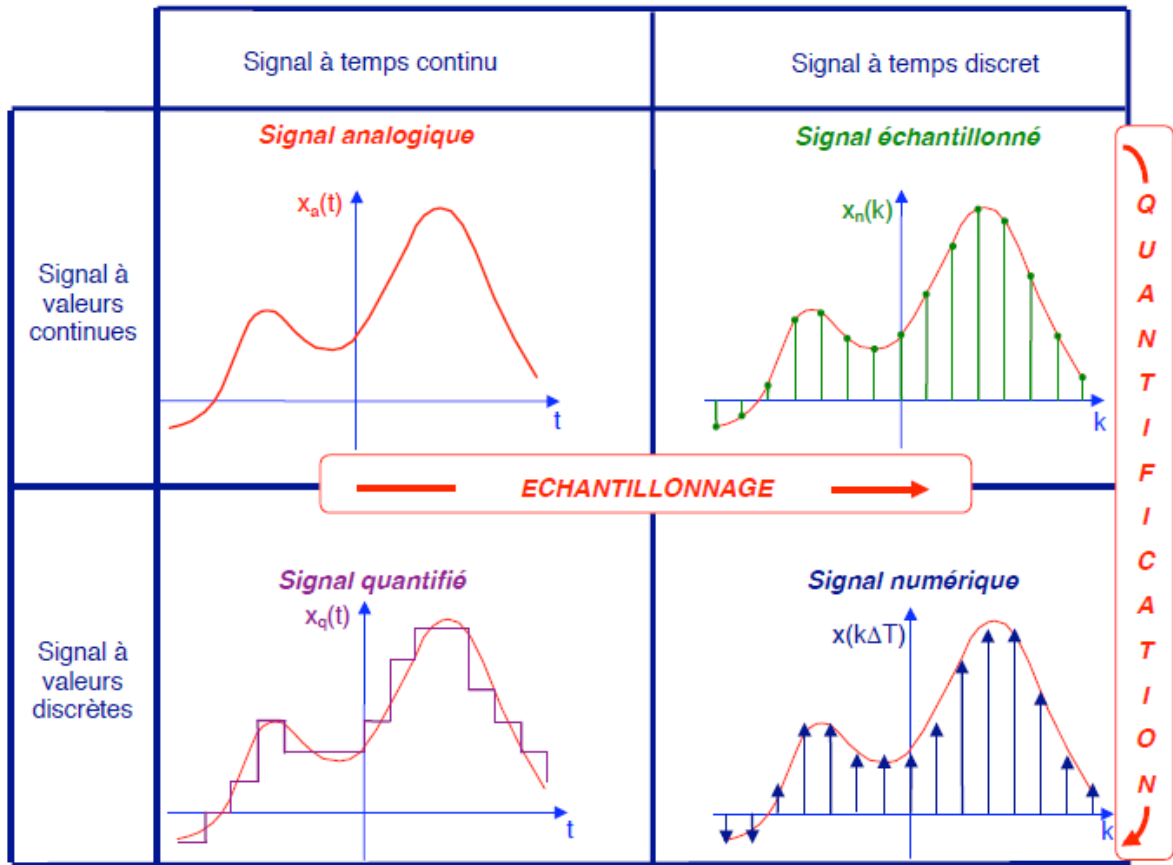
un échantillonnage aux points

$$x_k = kT, \quad k \in \mathbb{N},$$

nous donne les mêmes valeurs pour $f(x_k)$ et $g(x_k)$. On voit d'ailleurs assez facilement qu'un échantillonnage à une fréquence de moins de deux fois la fréquence maximale d'un signal ne peut pas le représenter.

5.2 Classification discret-continu

Le tableau suivant permet de comprendre l'enjeu de l'échantillonnage.



Considérons, sans restriction, un signal T -périodique quantifié (constant par morceaux) actif aux N dates (régulièrement distribuées)

$$t_0 = 0, \dots, t_k = \frac{kT}{N}, \dots, t_N = T.$$

Un tel signal s'écrit mathématiquement

$$f(t) = \sum_{k=1}^N \alpha_k I_{[t_k, t_{k+1}[}.$$

Notons qu'un tel signal n'est composé que de signaux élémentaires de période

- $\frac{T}{n}$ car le signal est T -périodique (cf le cours),
- $\frac{T}{n}$ avec $n \leq N$ sinon, le signal aurait des changements d'état entre les dates t_k .

Ainsi, la série de Fourier d'un tel signal ne verra des coefficients non nuls qu'entre $-N$ et N . Les c_{-k} étant redondants, on peut ne s'intéresser qu'au N coefficients $(c_n)_{0 \leq n \leq N}$.

Considérons que le signal numérique est composé d'impulsions α_k aux dates t_k sur une durée h . Mathématiquement, pour conserver l'énergie du signal, il est nécessaire de multiplier la valeur de

l'impulsion par $1/h$. Il s'agit de normaliser l'impulsion par sa durée. Un tel signal devient

$$f(t) = \sum_{k=1}^N \alpha_k \frac{\mathbf{I}_{[t_k, t_k+h[}}{h}.$$

d'où la notation de l'impulsion par une flèche.

5.3 Transformée de Fourier Discrète – Fast Fourier Transform (FFT)

Calculons les coefficients de Fourier du dernier signal (signal numérique) T -périodique,

$$c_k = \frac{1}{T} \int_0^T f(x) e^{\frac{-ik\pi}{T}x} dx.$$

Puisque

$$f(t) = \sum_{k=1}^N \alpha_k \frac{\mathbf{I}_{[t_k, t_k+h[}}{h},$$

Il suit que

$$\begin{aligned} c_n &= \frac{1}{hT} \sum_{k=1}^N \alpha_k \int_{t_k}^{t_k+h} e^{\frac{-in\pi}{T}x} dx \\ &= \frac{1}{hT} \sum_{k=1}^N \alpha_k \left[\frac{e^{\frac{-in\pi}{T}x}}{\frac{-in\pi}{T}} \right]_{t_k}^{t_k+h} \\ &= \frac{T}{-ihn\pi} \sum_{k=1}^N \alpha_k \left(e^{\frac{-in\pi}{T}(t_k+h)} - e^{\frac{-in\pi}{T}t_k} \right) \\ &= \frac{T}{-iThn\pi} \sum_{k=1}^N \alpha_k e^{\frac{-in\pi}{T}t_k} \left(e^{\frac{-in\pi}{T}h} - 1 \right) \\ &= \frac{1}{T} \frac{T \left(e^{\frac{-in\pi}{T}h} - 1 \right)}{-in\pi h} \sum_{k=1}^N \alpha_k e^{\frac{-in\pi}{T}t_k} \end{aligned}$$

D'après la limite bien établie

$$\lim_{x \rightarrow 0} \frac{e^x - 1}{x} = 1,$$

il suit, pour $h \rightarrow 0$,

$$\frac{T \left(e^{\frac{-in\pi}{T}h} - 1 \right)}{-in\pi h} \rightarrow 1.$$

On en déduit donc, en rappelant que $t_k = kT/N$,

$$c_n = \frac{1}{T} \sum_{k=1}^N \alpha_k e^{\frac{-in\pi}{T} \frac{kT}{N}}.$$

On définit donc la Transformée de Fourier Discrète (TFD) du signal numérique f par la suite

$$C_n = \frac{1}{T} \sum_{k=1}^N \alpha_k e^{\frac{-ink\pi}{N}},$$

pour $0 \leq k \leq N$. Des algorithmes rapides existent pour calculer ces coefficients de Fourier pour un signal représenté par les N valeurs de α_k . On parle de Fast Fourier Transform (FFT). C'est l'enjeu de notre TP.

5.4 Le signal

Sachant que l'oreille humaine perçoit les fréquences de 20 Hz à 20kHz, le théorème de Shannon montre qu'il suffit d'échantillonner le signal à une fréquence supérieure à 40kHz pour obtenir une restitution correcte du signal. On utilise souvent, pour des raisons techniques, une fréquence de 44,1 kHz pour les CD audio.

La voix humaine émet des sons à des fréquences comprises entre 0 et 1500 Hz. On échantillonne à 8KHz pour le téléphone. Posons H_e cette fréquence de référence, associée à la période d'échantillonnage

$$T_E = 1/8000 = 0,000125 \text{ s}$$

Soit 0,125 ms. Considérons le temps en ms et donnons la discrétisation du temps suivant d'une durée de 2 ms

```
discr=seq(0,2,0.125)
```

Définissons un signal naturel sur une discrétisation fine

```
discrn=seq(0,2,0.005)
signaln=sin(discrn*pi)+0.5*sin(discrn*3*pi)
        -2.5*sin(discrn*4*pi+9)+0.2*rnorm(length(discrn))
```

Notons que nous avons légèrement bruité notre signal grâce à `rnorm()`. Nous échantillonons toutes les 0,125 millisecondes, nous souhaitons donc récupérer la valeurs du signal `signaln` (de longueur 401) toutes les 25 valeurs.

```
signale=signaln[seq(1,401,25)]
```

Traçons le signal échantillonnée et le signal initial (analogique)

```
plot(discrn,signaln,type="l")    où l est un L minuscule
lines(discr,signale)
```

5.5 Fast Fourier Transform

Opérons la FFT

```
fft=fft(signale)
plot(discr,Mod(fft(signale)))
```

La liste `fft` contient les 17 coefficients de Fourier associés au signal échantillonné. Ils sont complexes. Ils sont donc difficiles à représenter. On peut néanmoins représenter le modules (Magnitude) pour avoir une idée de quelles fréquences sont présentes :

```
plot((0:16)/0.002,Mod(fft(signale)))
```

Ce graphe s'appelle l'analyse fréquentielle. Notons que l'on observe une symétrie très habituelle à la fréquence $F_e/2$.

5.6 Filtrage du signal

On souhaite éliminer les bruits du signal. Vu le graphe précédent, on peut neutraliser les fréquences dont le coefficient de Fourier a un module inférieur à 5.

```
fftf=fft  
fftf[Mod(fft)<5]=0
```

Comme la plupart des transformées, la FFT est réversible. C'est son grand intérêt (surtout si on n'a pas saisi toutes les nuances de la théorie). On va pouvoir recomposer notre signal filtré. Attention, la FFT n'est pas normalisée sur R. Il faut diviser par le nombre d'observations pour reconstituer un signal d'amplitude correct.

```
signalf=fft(fftf,inverse=TRUE)/length(discr)  
plot(discr,signalf,type="l",col="red")  
lines(discrn,signaln)
```

Part III

TP3 – Équations différentielles

6 Librairie R

Nous allons utiliser le package `deSolve` qui fait référence sur R pour le traitement des équations différentielles. On n'installera qu'une seule fois le package.

```
install.packages("deSolve")
```

Chargez la library `deSolve` :

```
library(deSolve)
```

7 Prise en main

7.1 L'équation

Nous nous proposons de résoudre le système de Cauchy suivant

$$(\mathcal{C}) = \begin{cases} y' &= ay(t-y), \\ y(0) &= y_0 \end{cases},$$

pour différentes valeurs de y_0 et du paramètre a .

7.2 Mise en mémoire de l'équation

L'équation différentielle de $(\mathcal{C})$ s'écrit $y' = f(t, y, a)$ avec

$$f(t, y, a) = ay(t-y).$$

Nous allons donc implémenter f

```
f=function(t,y,a){a*y*(t-y)}
```

7.3 Champ de directions

Il ne semble pas qu'il y ait de fonction native de R permettant de tracer le champ de directions associé à une équation différentielle. C'est néanmoins un graphe assez simple dès lors qu'on connaît la fonction `arrow(x0,y0,x1,y1,length=0.05)` qui rajoute une flèche de (x_0, y_0) vers (x_1, y_1) sur un graphe existant (avec une envergure `length` correcte). Ainsi, en se rappelant que la flèche d'un champ de direction doit avoir pour pente la valeur de f prise en son centre, on peut, sur un graphe vide (ou presque) ajouter la direction issue de $(1; 0.5)$:

```

a=0.25
x=c(-5,5)
plot(x,x)
arrows(1-0.1, 0.5-0.1*f(1,0.5,a),1+0.1, 0.5+0.1*f(1,0.5,a),length=0.05)

```

où la longueur horizontale de notre flèche sera $2 \times 0,1$. Nous avons compris le principe, mais nous aimerions remplir le plan de telles flèches. Passons par une fonction champs.

```

champ=function(xmax,pas,f,a){          ||max axes/pas/fonction/paramètre de la fonction
x=c(-xmax,xmax)
plot(x,x)                             ||graphe vide
discr=seq(-xmax,xmax,pas)              ||discrétisation des axes
points=expand.grid(discr,discr)        ||ensemble des points du plan
arrows(points[,1]-pas/5, points[,2]-pas/5*f(points[,1],points[,2],a),
        points[,1]+pas/5, points[,2]+pas/5*f(points[,1],points[,2],a),length=0.05)
}

```

Spécifions des valeurs arbitraires de a (`a=...`) et appelons

```
champ(5,0.5,f,a)
```

Votre fonction `champ` vous servira dans la suite.

7.4 Solution de $\mathcal{C}$

La librairie `deSolve` nous donne des solutions *numériques*, c'est à dire calculées de proche en proche, de nos équations. D'un point de vue technique, elle a besoin, dans ses arguments, d'une fonction qui rend une liste et non, une valeur comme `f` le fait. Ainsi, et c'est un peu pénible, nous devons transformer les sorties de `f` en liste par

```
Eq=function(t,y,a){list(f(t,y,a))}
```

Il reste à demander la solution souhaitée avec `ode` :

```

y0=2          ||condition initiale
a=0.25        ||valeur du paramètre
discr = seq(from = 0, to = 5, by = 0.02)           || discrétisation de l'intervalle de temps
sol=ode(y = y0, times = discr, func = Eq, parms = a)
plot(sol)

```

7.5 Variations de y_0

Posons

```
a=0.25
```

Dans le plan $[-5, 5] \times [-5, 5]$ muni du champs de vecteurs,

```
champ(5,0.5,f,a)
```

nous pouvons observer des trajectoires de solutions pour différentes valeurs initiales et, en particulier, les solutions associées aux conditions initiales -1, 1, 2 :

```
discr = seq(0,5,0.02)
sol1=ode(y = 1, times = discr, func = Eq, parms = a)
sol2=ode(y = 2, times = discr, func = Eq, parms = a)
solm1=ode(y = -1, times = discr, func = Eq, parms = a)
```

Tant pis pour le message d'erreur éventuel. Ajoutons maintenant les tracés

```
lines(sol1)
lines(sol2)
lines(solm1)
```

7.6 Solutions issues de $y_0 = 1$ en fonction des valeurs de a

Posons la condition initiale $y_0 = 1$

```
y0=1
```

Dans le plan $[-5, 5] \times [-5, 5]$,

```
plot(c(-5,5),c(-5,5))
```

Observons la trajectoire de condition initiale $y_0 = 1$ pour des valeurs de $a = 1; a = 0.25; a = -0.25$

```
discr = seq(0,5,0.02)
sol1=ode(y = y0, times = discr, func = Eq, parms = 1)
sol2=ode(y = y0, times = discr, func = Eq, parms = 0.25)
sol3=ode(y = y0, times = discr, func = Eq, parms = -0.25)
```

Ajoutons maintenant les tracés

```
lines(sol1)
lines(sol2)
lines(sol3)
```

8 Les modèles de croissance de population

On se propose de regarder les équations liées à différents modèles de croissance de population. Le champ d'application de ces modèles va au delà, en particulier, en économie, dans les problèmes de durée de vie d'un produit...

8.1 Modèle de Malthus

On considère, sans perte de généralité, une population de taille initiale 1 (on considère une évolution en termes de proportion de la taille de la population initiale) modélisée par le modèle de croissance exponentiel de Malthus :

$$y' = ry,$$

où r est le paramètre lié aux taux de mortalité et de natalité.

On observe immédiatement que

- Si $r < 0$, la population s'éteint,
- Si $r > 0$, la population explose.

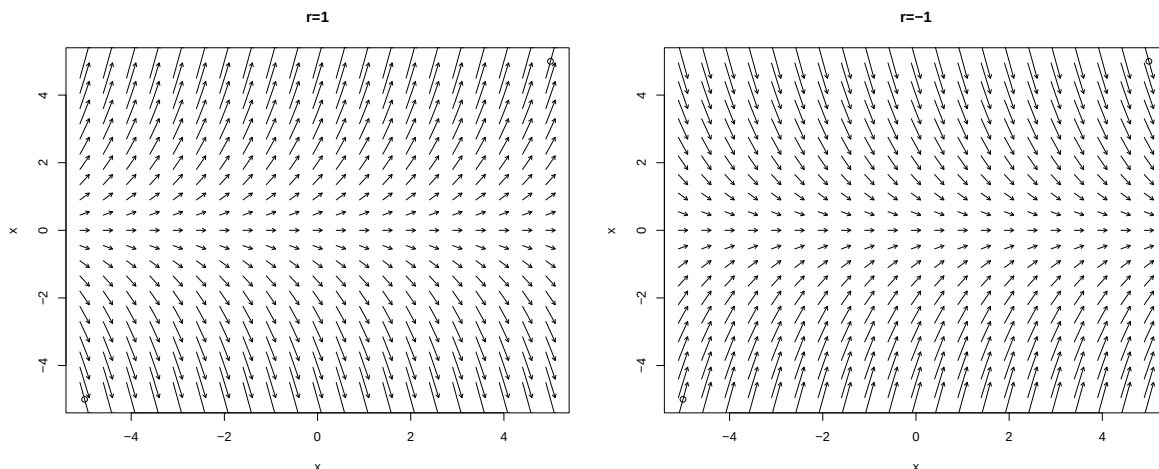
En effet, posons

```
f=function(t,y,r){r*y}
Eq=function(t,y,r){list(f(t,y,r))}
y0=1
```

Partageons la fenêtre graphique en deux et collons côte à côte les champs de directions :

```
par(mfrow=c(1,2))
champ(5,0.5,f,1)
title("r=1")
champ(5,0.5,f,-1)
title("r=-1")
```

et le résultat

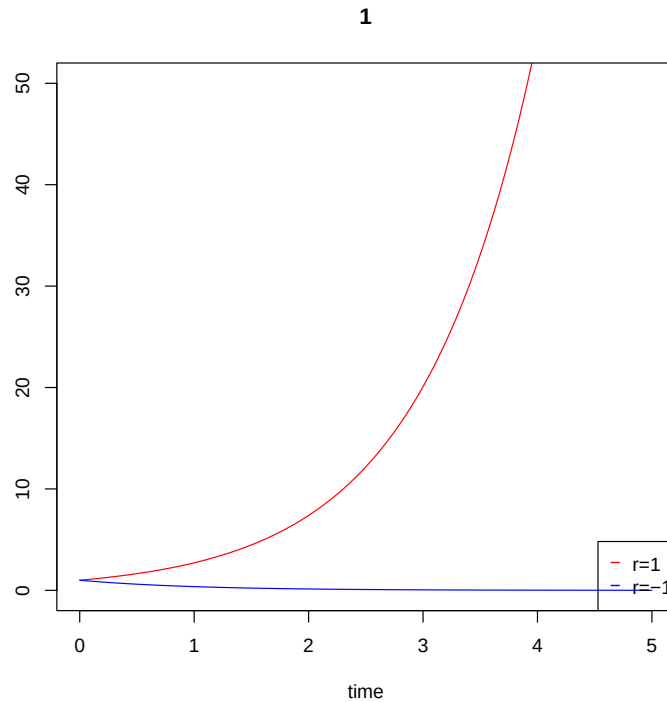


Traçons les solutions issues de $y_0 = 1$ sur le même graphe avec les valeurs de r précédentes.

```

discr = seq(0,5,0.02)
sol1=ode(y = y0, times = discr, func = Eq, parms = 1)
solm1=ode(y = y0, times = discr, func = Eq, parms = -1)
plot(sol1,ylim=c(0,50),col="red")
lines(solm1,col="blue")
legend(x="bottomright",legend=c("r=1","r=-1"), col=c("red","blue"),pch=c("-", "-"))

```



8.2 Le modèle logistique de Verhulst

On considère une population de taille initiale 1 modélisée par le modèle logistique de Verhulst :

$$y' = ry \left(1 - \frac{y}{K}\right),$$

où r est le paramètre lié à la croissance (positive ou négative) de la population et K la limite de la population si $r > 0$. Nous allons l'observer.

En effet, posons

```

f=function(t,y,a){a[1]*y*(1-y/a[2])}
Eq=function(t,y,a){list(f(t,y,a))}
y0=1

```

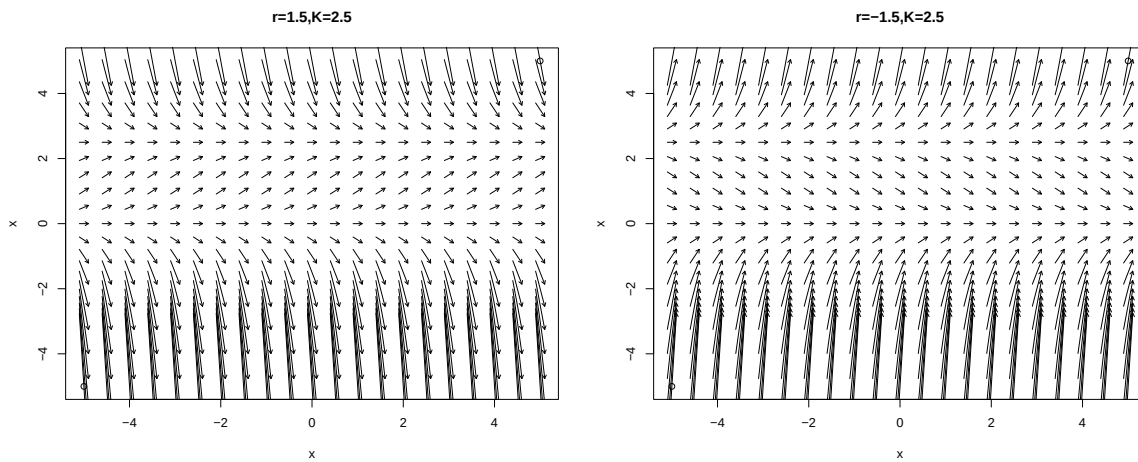
Notons que le paramètre est un vecteur à deux composantes. Posons les couples de paramètres (r, K) suivants

```
a1=c(1.5,2.5)
a2=c(-1.5,2.5)
```

Partageons la fenêtre graphique en deux et collons côte à côte les champs de directions :

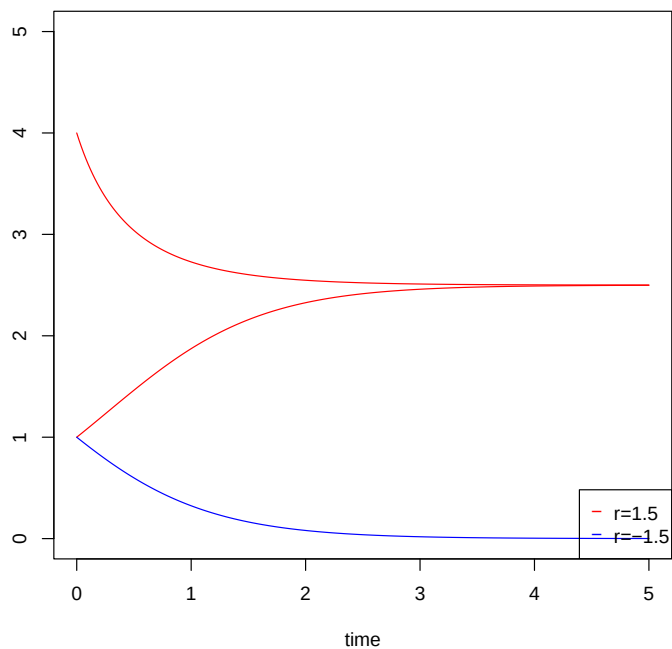
```
par(mfrow=c(1,2))
champ(5,0.5,f,a1)
title("r=1.5,K=2.5")
champ(5,0.5,f,a2)
title("r=-1.5,K=2.5")
```

et le résultat



Traçons les solutions issues de $y_0 = 1$ sur le même graphe avec les valeurs de paramètres précédentes.

```
discr = seq(0,5,0.02)
sola1=ode(y = y0, times = discr, func = Eq, parms = a1)
sola1bis=ode(y = 4, times = discr, func = Eq, parms = a1)
sola2=ode(y = y0, times = discr, func = Eq, parms = a2)
plot(sola1,ylim=c(0,5),col="red")
lines(sola1bis,col="red")
lines(sola2,col="blue")
legend(x="bottomright",legend=c("r=1.5","r=-1.5"), col=c("red","blue"),pch=c("-", "-"))
```



9 Travail à rendre

Vous rendrez sur une feuille unique enregistrée en `.pdf`, un envoi par groupe, dans un mail co-signé à l'adresse `julien.grepat@univ-grenoble-alpes.fr` le travail suivant.

9.1 Pêche sans quota

On considère la dynamique de population d'une certaine espèce de poisson pêchable sans quota donnée par

$$y' = ry \left(1 - \frac{y}{K}\right) - Ey^2,$$

où E est l'effort de pêche. On supposera que $y_0 = 1$, $K = 5$ (autrement dit, la population aujourd'hui est 20% de ce qu'elle devrait être), et $r < 0.1$ que vous choisirez arbitrairement.

La première ligne de votre travail comprendra trois graphes légendés avec les valeurs des paramètres :

- le champ de directions pour $E1 = 0.2$,
- le champ de directions pour $E2 = 0.9$,
- les solutions de l'équation pour ces deux cas.

9.2 Le modèle de pêche par quota

On considère maintenant que la dynamique de population de cette espèce de poisson pêchable avec quota est donnée par

$$y' = ry \left(1 - \frac{y}{K}\right) - Q,$$

où Q est le quota. On supposera que $y_0 = 1$, $K = 5$, et vous choisirez $0,4 < r < 0,6$ que vous choisirez arbitrairement.

Donnez un quota Q pour lequel la population diminue, un autre pour lequel la population augmente.

La deuxième ligne de votre travail comprendra trois graphes légendés avec les valeurs des paramètres :

- le champ de directions pour la première valeur de Q ,
- le champ de directions pour la seconde valeur de Q ,
- les solutions de l'équation pour ces deux cas.